Brandt Shirley

4/4/26

IECE 111

Dave Ardrey

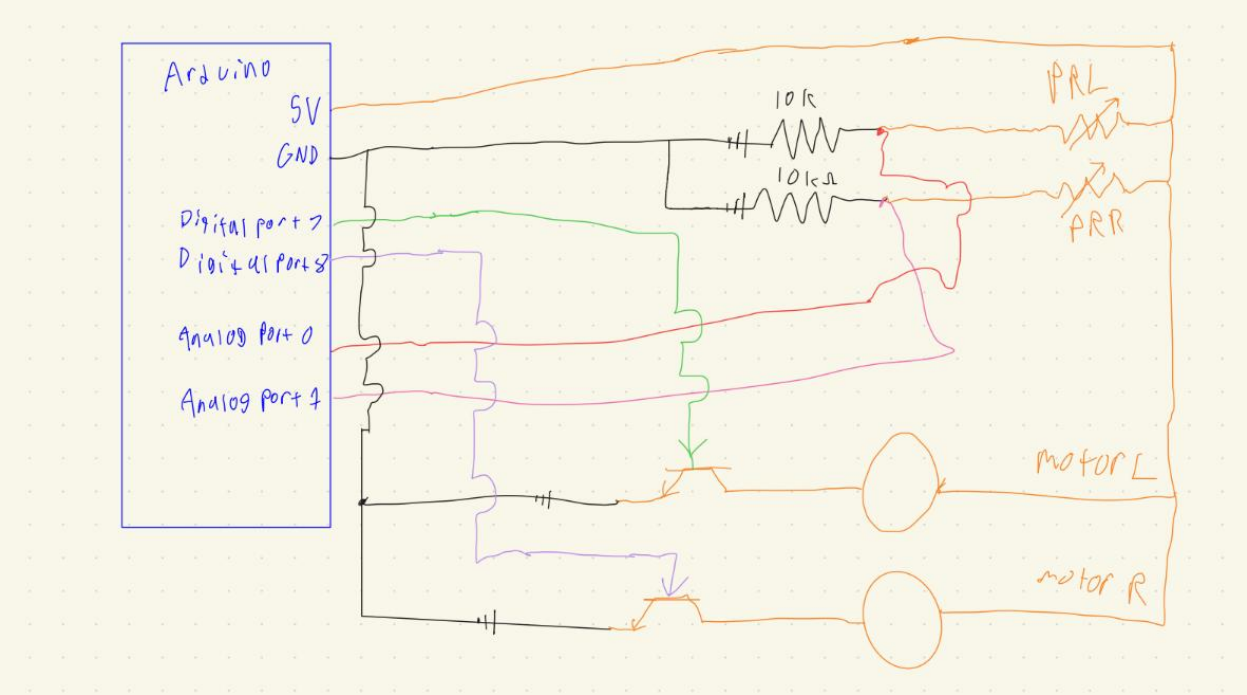
Project 2 Report

Summary

The final state of my line follower was successful as the line follower was able to follow all three tracks and stop at the finish line. There were a few problems I ran into trying to get it to this point, with the first one being setting up the motor and photoresistor together as the photoresistor would not be set up correctly and would not give the correct readings, thus not letting the motor turn on and when the photoresistor was setup correctly the motor was wired incorrectly and wouldn't turn on. To fix it I just had to rewire the photoresistor circuits to make it run correctly and make sure that both the photoresistor circuits and the motor circuits were wired correctly and had the correct digital and analog pins wired to them. The second problem I ran into was that the line follower had the incorrect cutoff resistance so even when it saw the black line, the resistance was barely under the threshold and would continue to run that motor and run past the line. To fix this I just rewrote the cutoff resistance to a smaller number, making the robot have less wiggle room and would react to the tape more effectively but not to low to where the shadows would stop the motor. The third problem was that the line follower was running around the finish line because the photoresistors were set up too far apart so with one of the motors being barely stronger than the other, the line follower would stay on one side of the line and maneuver around the finish line. To fix this I just set the photoresistors closer together so that they wouldn't miss

the finish line and would stay in the track on a tighter line. The fourth problem I ran into was that the line follower was moving too far forward and on the final track, missing the line. To fix this I changed the delay in the code to allow the motors to be on for 50ms instead of 80ms making the moves smaller than the width of the tape and in the worst case, stopping it on the line to read it. The functionality of the line follower was that it had 2 photoresistors that were in a circuit so that once it saw a dark setting it gave a high resistance and a low voltage out and in a bright setting it gave a low resistance and high voltage out. Using this I could tell when I saw the line or not and with this info if the left sensor saw the line, the left wheel would turn off for that cycle and the right wheel would turn on to make the car turn right. When the right sensor saw the line, the right wheel would turn off, and the left wheel would turn on and make the car turn left. If neither sensor saw the line, then both would turn on and move the car forward and if both saw the line then both would turn off and the car would stop. Some tests we ran on the line follower was seeing if it worked on a straight line, see if it worked on a curved line, and see if it worked on a track that had sharp turns. In the first test it had some issues but with some minor tweaks I got it working on that, on the second test it worked flawlessly, and on the third test the line follower had some slight issues but was able to work after some more tweaks. During these tests I had the line follower work slow so there was less room for mistakes and so it could obtain clear readings.

Circuit Schematic



Circuit Description

In this circuit we will start off with the analog ports for the photoresistor, where we have one side connected to ground and that goes through a 10k resistor so that we create a voltage divider circuit. With this we can use the second side of the circuit that is connected to the voltage in and to a photo resistor where once the resistor is dark the equation will look like this $(\frac{10k}{10k+500k} * 5 = .1V)$ and we will have a high resistance and get a small voltage value, where when the resistor is bright the equation will look like this $(\frac{10k}{10k+20k} * 5 = 1.66V)$ and we will see a low resistance and get a high voltage value. With the values that go into the analog ports we can use the voltage divider equation to calculate the resistance of the photo resistor and use a threshold to say if the photo resistor sees a bright or dark surface. With this we can move to the motors and digital ports where we see the motor is connected with one end to the voltage in and

the other side is connected to the ground but there is a NMOS transistor in the way so if we determine the surface as dark the digital port will give a 0 and will make the gate act as a open switch, not allowing the motor to connect to ground and not turn on, when we see a bright surface we will send a 1 through the digital port and will make the gate act as a wire, allowing the motor to connect to ground and turn on.

Pseudocode

//Sets Digital pin 7 to the left motor and Digital pin 8 to the right motor

Setup where we are reading motor left and right for digital pins

//Sets Analog pin 0 to the left photoresistor and Digital pin 1 to the right photoresistor

Setup where we are reading motor left and right for analog pins

//Sets int values for ADValL and ADValR to contain the readings from the photoresistors

Setup the ADValues that we will read for the left and right motors

//Creates a int called Vin and sets it to 5

Set Vin from the Arduino

//Sets float values called VoutL and VoutR to contain the voltage from ADVal/1024.0

Setup a float for the voltage out from the left and right motors photoresistor circuits

//Sets a float called R2 which stores the resistor from the photoresistor circuits (10k)

Set the Resistors values in the photoresistor circuit to calculate the resistance of the photoresistor using the voltage divider equation.

//Sets floats that will store the resistance from the left and right photoresistors

Set float values that will store the value of the left and right photoresistor resistance

//Sets float values that will store a temporary value during the voltage divider equation so that no values are overwritten

Set a float value for buffer that will store a temporary value for the voltage divider equation

//go into void setup()

Setup circuit

//use pinMode to set the pin values stored before to input for the analog pins and output for the digital pins

Set the digital pins for motor left and right for output and the analog pins for motor left and right for input

//Enters void loop()

Go into the loop

//uses analog read to read the ADvalues from the left and right analog pins and stores them in ADvalL and ADvalR

Read the values from the left and right analog pins

//Checks that both analog pins got a reading

If both have value that isn't 0, go to do calculations

//Prints the ADval of the left photoresistor and then does the voltage divider equation for the left
//photoresistor

Output ADval of the left photoresistor and do voltage divider equation for left motor

//outputs the voltage that we stored for VoutL using ADValL/1024.0 and the resistance we
//calculated for the left photoresistor

Output the Voltage out for the left photoresistor and the resistance of the left photoresistor

//Prints the ADval of the right photoresistor and then does the voltage divider equation for the
//right photoresistor

Output ADval of the right photoresistor and do voltage divider equation for right motor

//outputs the voltage that we stored for VoutR using ADValR/1024.0 and the resistance we
//calculated for the right photoresistor

Output the Voltage out for the right photoresistor and the resistance of the right photoresistor

//uses if(R1L <2700 && R1R<2700) to see if they are seeing a bright surface and if so uses
//digitalWrite to set both digital pins for the motors to high to connect them to ground, waits
//50ms, then sets both digital pins to LOW so that the motors turn off

Check if the resistance of both photoresistors is less than 2700 and if so, turn on both motors for
50ms and then turn them both off

//uses ifelse(R1L <2700) to see if the left photoresistor is seeing a bright surface and if so, uses
//digitalWrite to set digital pin 7 for the left motor to high to connect them to ground, waits
//50ms, then sets digital pin 7 to LOW so that the left motor turn off

If the last check was false then check if the resistance of the left photoresistor is less than 2700 and if so, turn on the left motor for 50ms and then turn the left motor off

//uses ifelse(R1R <2700) to see if the right photoresistor is seeing a bright surface and if so, uses //digitalWrite to set digital pin 8 for the right motor to high to connect them to ground, waits //50ms, then sets digital pin 8 to LOW so that the right motor turn off

If the last check was false then check if the resistance of the right photoresistor is less than 2700 and if so, turn on the right motor for 50ms and then turn the right motor off

//uses delay(300) to wait 300ms then goes back through the loop

Delay for 300ms before reading both photoresistors again and repeating the loop

Testing

For testing I first checked with part 2 if the photoresistor would give good outputs before moving on by exposing it to light and darkness and checking the outputs, finding no problems. Next, I checked the 3rd activity to see if the motor would work with the photoresistor and to set a good threshold, finding no problems. Then, I checked on the 4th activity to make sure that the left and right photoresistor and motor value were correct and that they would work independently and for the correct sides, finding no problems. Finally, for part 5 I slowly worked up the 3 tracks to find any problems and resolve them. On the first track I saw that the line follower drove to the right because the left motor was a bit stronger than the right motor which wasn't a issue but the line follower drove off the track so to resolve it I changed the threshold to a smaller value so that the motors would not drive off the track but also not stop in their shadow. After that it made it to the

finish line but drove around the finish line because the photoresistors were too far apart so to fix this I just make the photoresistors closer together so this couldn't happen. After this it completed track 1 and 2 with no issues but on track 3 it drove over a turn and missed the line so to fix it I just set the time the motor was on to 50ms instead of 80ms so it couldn't drive past the line and after that it completed the final track and the project.